



Universidad Mariano Gálvez de Guatemala

Facultad de Ingeniería en Sistemas de Información y Ciencias de la Computación

# Sistemas Operativos II

Proyecto: K Monitor

Docente: Ing. René Ornelis

junio de 2026

## Proyecto grupo 10



### integrantes del grupo:

Mynor Antonio Ramirez Bautista  
Andres Marcelo Rodas Cuevas  
Alvin Ricardo Barillas Herrarte

7690-24-205  
9490-13-7424  
3190-10-10997



## ÍNDICE

|                                                                  |           |
|------------------------------------------------------------------|-----------|
| <b>Carátula e Integrantes .....</b>                              | <b>1</b>  |
| <b>1. Introducción .....</b>                                     | <b>2</b>  |
| <b>2. Marco Teórico .....</b>                                    | <b>3</b>  |
| <b>2.1 Kernel Linux .....</b>                                    | <b>3</b>  |
| <b>3. Arquitectura de la Solución .....</b>                      | <b>5</b>  |
| <b>3.1 Descripción General .....</b>                             | <b>5</b>  |
| <b>3.2 Componentes de la Arquitectura .....</b>                  | <b>6</b>  |
| <b>3.3 Flujo de Funcionamiento .....</b>                         | <b>8</b>  |
| <b>3.4 Diagrama de Arquitectura .....</b>                        | <b>10</b> |
| <b>3.5 Ventajas de la Solución .....</b>                         | <b>11</b> |
| <b>4. Descripción del Contenido de los Archivos Fuente .....</b> | <b>12</b> |
| <b>4.1 Archivo kmonitor.c .....</b>                              | <b>12</b> |
| <b>4.2 Archivo kmonitor.h .....</b>                              | <b>18</b> |
| <b>4.3 Archivo Makefile .....</b>                                | <b>19</b> |
| <b>5. Estructuras de Datos del Kernel Utilizadas .....</b>       | <b>20</b> |
| <b>5.1 Estructura task_struct .....</b>                          | <b>20</b> |
| <b>5.2 Macro for_each_process() .....</b>                        | <b>22</b> |
| <b>5.3 Estructura sysinfo .....</b>                              | <b>23</b> |
| <b>5.4 Sistema seq_file .....</b>                                | <b>24</b> |
| <b>5.5 Estructura proc_ops .....</b>                             | <b>25</b> |
| <b>5.6 Estructura proc_dir_entry .....</b>                       | <b>26</b> |
| <b>5.7 Relación entre las Estructuras Utilizadas .....</b>       | <b>27</b> |
| <b>6. Conclusiones Generales .....</b>                           | <b>28</b> |
| <b>7. Recomendaciones .....</b>                                  | <b>29</b> |
| <b>8. Bibliografía .....</b>                                     | <b>29</b> |



## INTRODUCCIÓN

Los sistemas operativos modernos constituyen la capa fundamental que permite la interacción entre el hardware y las aplicaciones utilizadas por los usuarios. Dentro de estos sistemas, el kernel o núcleo desempeña un papel esencial al encargarse de la administración de recursos, gestión de procesos, control de memoria y comunicación con los dispositivos físicos del equipo.

Linux es uno de los sistemas operativos más utilizados en servidores, supercomputadoras y entornos empresariales debido a su estabilidad, seguridad y naturaleza de código abierto. Una de sus características más importantes es la capacidad de extender sus funcionalidades mediante Módulos Cargables del Kernel (Loadable Kernel Modules o LKM), los cuales permiten incorporar nuevas características sin necesidad de recompilar o reiniciar el sistema operativo.

El presente proyecto denominado K-Monitor consiste en el desarrollo de un módulo cargable para Linux capaz de obtener información relacionada con la memoria del sistema y los procesos activos. La información es recolectada directamente desde las estructuras internas del kernel y posteriormente expuesta al usuario a través del sistema de archivos virtual /proc.

La implementación de este proyecto permite comprender conceptos fundamentales relacionados con la arquitectura interna del sistema operativo, la administración de memoria, la gestión de procesos y el funcionamiento de los módulos del kernel. Asimismo, proporciona experiencia práctica en el desarrollo de componentes de bajo nivel que interactúan directamente con el núcleo del sistema operativo.



## MARCO TEÓRICO

### Kernel Linux

El Kernel Linux es el componente central del sistema operativo Linux. Su principal responsabilidad consiste en administrar los recursos de hardware y proporcionar servicios a las aplicaciones que se ejecutan en el sistema.

Entre sus funciones principales se encuentran:

- Administración de procesos.
- Gestión de memoria.
- Control de dispositivos.
- Administración de sistemas de archivos.
- Gestión de seguridad y permisos.
- Comunicación entre hardware y software.

El kernel actúa como intermediario entre las aplicaciones de usuario y los recursos físicos del equipo.

### Arquitectura de la Solución

#### Descripción General

K-Monitor es un Módulo Cargable del Kernel (Loadable Kernel Module - LKM) desarrollado en lenguaje C para Linux. Su función principal es obtener información directamente desde las estructuras internas del kernel relacionadas con la memoria del sistema y la administración de procesos, exponiendo dichos datos al espacio de usuario mediante una entrada dentro del sistema de archivos virtual /proc.

La arquitectura implementada sigue el modelo tradicional de interacción entre el espacio de usuario (User Space) y el espacio del kernel (Kernel Space), permitiendo consultar métricas del sistema en tiempo real sin necesidad de modificar permanentemente el núcleo del sistema operativo.



## Componentes de la Arquitectura

### Módulo del Kernel

El archivo principal kmonitor.c contiene toda la lógica del módulo. Este componente es responsable de:

- Inicializar el módulo cuando se ejecuta insmod.
- Crear el archivo /proc/kmonitor\_Grupo\_10.
- Obtener estadísticas de memoria mediante estructuras internas del kernel.
- Recorrer la tabla global de procesos activos.
- Mostrar la información al usuario cuando se lee el archivo /proc.
- Liberar recursos al descargar el módulo mediante rmmod.

### Archivo de Cabecera

El archivo kmonitor.h centraliza las librerías del kernel necesarias para el funcionamiento del proyecto.

Este archivo proporciona acceso a:

- Interfaces para módulos del kernel.
- Gestión de procesos.
- Gestión de memoria.
- Sistema de archivos /proc.
- Funciones de lectura secuencial mediante seq\_file.

### Sistema de Archivos Proc

El directorio /proc es un sistema de archivos virtual administrado por el kernel.

Su función es exponer información interna del sistema operativo como si fueran archivos normales.

Cuando el usuario ejecuta:



cat /proc/kmonitor\_Grupo\_10

el kernel invoca las funciones definidas por el módulo para generar dinámicamente la información mostrada.

### **Subsistema de Memoria**

La información de memoria es obtenida utilizando la estructura sysinfo y la función:

si\_meminfo()

Esta función permite consultar:

- Memoria total.
- Memoria libre.
- Memoria utilizada.
- Porcentaje de uso.

La información es obtenida directamente desde la Unidad de Administración de Memoria (MMU) administrada por el kernel.

### **Subsistema de Procesos**

Para obtener la lista de procesos activos se utiliza la macro:

for\_each\_process()

Esta macro recorre la lista global de tareas del sistema.

Por cada proceso se consulta:

- PID
- Nombre
- Estado



La información es obtenida desde la estructura `task_struct`.

## **Flujo de Funcionamiento**

### **Fase 1: Carga del Módulo**

Cuando el administrador ejecuta:

```
insmod kmonitor.ko
```

ocurre lo siguiente:

1. El kernel carga el módulo en memoria.
2. Se ejecuta la función `iniciar_kmonitor()`.
3. Se crea la entrada `/proc/kmonitor_Grupo_10`.
4. El módulo queda listo para responder consultas.

### **Fase 2: Lectura de Información**

Cuando el usuario ejecuta:

```
cat /proc/kmonitor_Grupo_10
```

ocurre lo siguiente:

1. El sistema abre el archivo virtual.
2. Se ejecuta la función `abrir_kmonitor()`.
3. Se llama a `mostrar_kmonitor()`.
4. Se recopilan las estadísticas de memoria.
5. Se recorre la tabla de procesos.
6. La información es enviada al buffer secuencial.
7. El contenido es mostrado al usuario.

## Fase 3: Descarga del Módulo

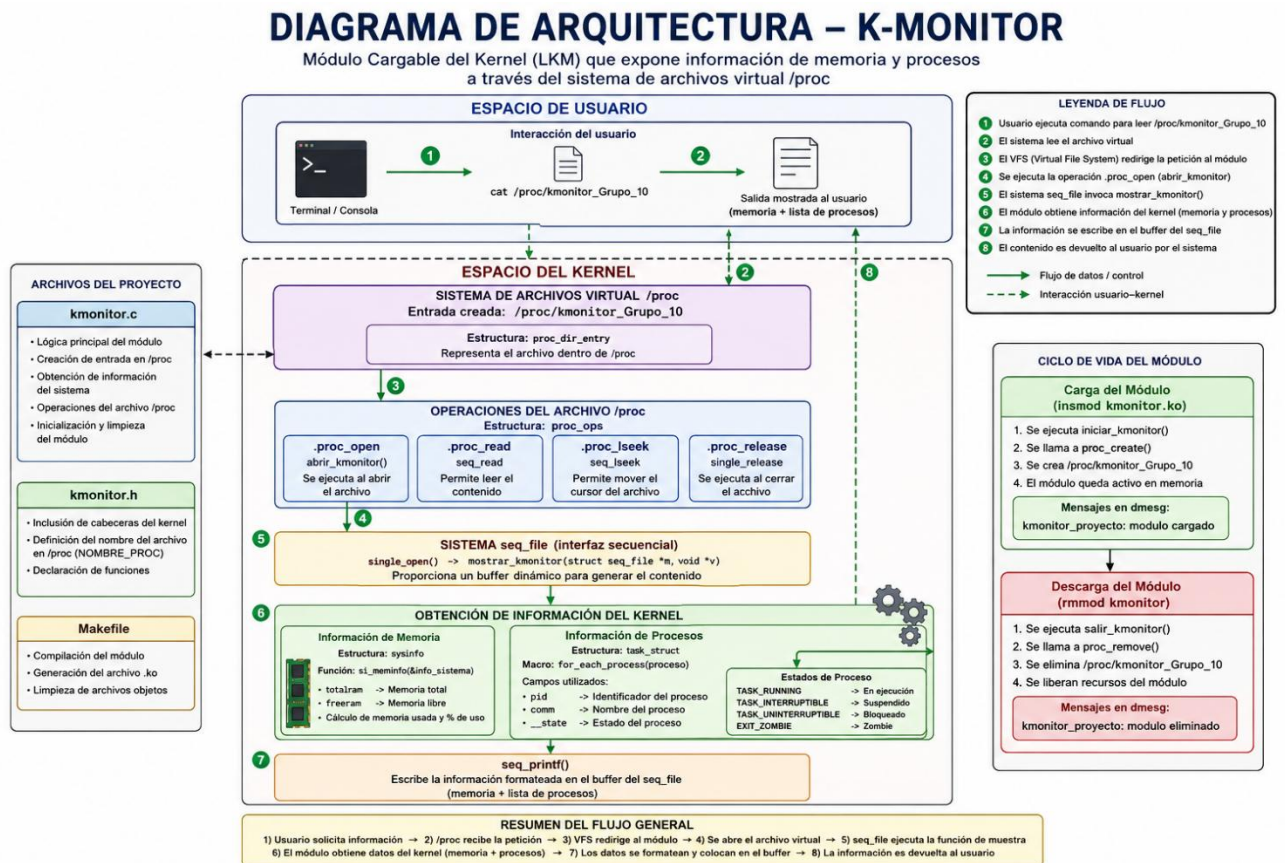
Cuando se ejecuta:

`rmmod kmonitor`

ocurre lo siguiente:

1. Se ejecuta la función `salir_kmonitor()`.
2. Se elimina el archivo `/proc/kmonitor_Grupo_10`.
3. Se liberan los recursos asociados.
4. El módulo desaparece completamente del kernel.

## Diagrama de Arquitectura







## **Ventajas de la Solución**

- No modifica el kernel permanentemente.
- Permite monitoreo en tiempo real.
- Utiliza interfaces oficiales del kernel.
- Consume pocos recursos.
- Facilita la comprensión de la administración interna de Linux.
- Demuestra la interacción entre procesos, memoria y módulos del kernel.

## **Descripción del Contenido de Cada Archivo Fuente**

### **Archivo: kmonitor.c**

#### **Descripción General**

El archivo kmonitor.c contiene la implementación principal del módulo cargable del kernel. Su responsabilidad es obtener información sobre la memoria y los procesos del sistema operativo Linux, para posteriormente mostrarla al usuario mediante una entrada en el sistema de archivos virtual /proc.

Este archivo implementa todas las funciones necesarias para:

- Inicializar el módulo.
- Crear la entrada en /proc.
- Obtener información de memoria.
- Recorrer la tabla de procesos.
- Mostrar información al usuario.
- Liberar recursos al descargar el módulo.

#### **Inclusión del Archivo de Cabecera**

```
#include "kmonitor.h"
```



Esta línea importa el archivo de cabecera del proyecto, donde se encuentran definidas todas las bibliotecas necesarias del kernel y la constante que define el nombre del archivo dentro de /proc.

### **Información del Módulo**

```
MODULE_LICENSE("GPL");
```

Define la licencia GPL para el módulo.

Permite que el kernel considere el módulo como software libre y le otorgue acceso a símbolos internos que no están disponibles para módulos propietarios.

```
MODULE_AUTHOR("Grupo Proyecto");
```

Indica el autor o grupo desarrollador del módulo.

Esta información puede visualizarse utilizando herramientas de administración de módulos.

```
MODULE_DESCRIPTION("Modulo de monitoreo del sistema - Proyecto");
```

Proporciona una descripción textual del propósito del módulo.

```
MODULE_VERSION("1.0");
```

Especifica la versión actual del módulo.

### **Variable Global**

```
static struct proc_dir_entry *entrada_proc;
```

Declara un puntero que almacenará la referencia al archivo creado dentro del sistema /proc.



Esta referencia será utilizada posteriormente para eliminar el archivo cuando el módulo sea descargado.

### **Función estado\_proceso()**

```
static const char *estado_proceso(long estado)
```

Esta función recibe el valor numérico del estado interno de un proceso y devuelve una cadena de texto legible para el usuario.

Su objetivo es traducir los estados internos del kernel a nombres comprensibles.

#### **Estado Running**

```
if (estado == TASK_RUNNING)
    return "En ejecucion";
```

Indica que el proceso se encuentra ejecutándose o listo para ejecutarse.

#### **Estado Interruptible**

```
else if (estado & TASK_INTERRUPTIBLE)
    return "Suspendido";
```

Indica que el proceso está esperando un evento y puede ser despertado mediante una señal.

#### **Estado No Interruptible**

```
else if (estado & TASK_UNINTERRUPTIBLE)
    return "Bloqueado";
```

Indica que el proceso espera un recurso crítico y no puede ser interrumpido.

#### **Estado Zombie**



```
else if (estado == EXIT_ZOMBIE)
    return "Zombie";
```

Representa un proceso terminado cuyo proceso padre aún no ha leído su estado de salida.

### **Estado Desconocido**

```
else
    return "Desconocido";
```

Captura cualquier estado que no coincida con las condiciones anteriores.

### **Función mostrar\_kmonitor()**

```
static int mostrar_kmonitor(struct seq_file *m, void *v)
```

Es la función principal del proyecto.

Se ejecuta cada vez que un usuario consulta:

```
cat /proc/kmonitor_Grupo_10
```

Su función es generar toda la información que será mostrada.

### **Declaración de Variables**

```
struct sysinfo info_sistema;
```

Estructura utilizada para almacenar información general de memoria del sistema.

```
struct task_struct *proceso;
```

Puntero utilizado para recorrer la lista de procesos del sistema.



```
unsigned long memoria_total;  
unsigned long memoria_libre;  
unsigned long memoria_usada;
```

Variables destinadas al almacenamiento de estadísticas de memoria.

```
int porcentaje_uso;
```

Variable utilizada para calcular el porcentaje de memoria utilizada.

### **Obtención de Información de Memoria**

```
si_meminfo(&info_sistema);
```

Solicita al kernel que llene la estructura sysinfo con información actual de memoria.

```
memoria_total = info_sistema.totalram * 4;
```

Calcula la memoria total disponible en kilobytes.

```
memoria_libre = info_sistema.freeram * 4;
```

Calcula la memoria libre disponible.

```
memoria_usada = memoria_total - memoria_libre;
```

Calcula la memoria actualmente utilizada.

```
porcentaje_uso = (memoria_total) ? (memoria_usada * 100) / memoria_total : 0;
```

Calcula el porcentaje de utilización de memoria.

### **Impresión de Información de Memoria**

```
seq_printf(m, "===== MEMORIA DEL SISTEMA =====\n");
```

Muestra el encabezado de la sección de memoria.



```
seq_printf(m, "Memoria total: %lu KB\n", memoria_total);
```

Imprime la memoria total.

```
seq_printf(m, "Memoria libre: %lu KB\n", memoria_libre);
```

Imprime la memoria libre.

```
seq_printf(m, "Uso de memoria: %d%%\n\n", porcentaje_uso);
```

Imprime el porcentaje de uso de memoria.

### Listado de Procesos

```
seq_printf(m, "==== LISTA DE PROCESOS =====\n");
```

Imprime el encabezado de la tabla de procesos.

```
seq_printf(m, "%-10s %-25s %-15s\n", "PID", "NOMBRE", "ESTADO");
```

Imprime los nombres de las columnas.

### Recorrido de Procesos

```
for_each_process(proceso)
```

Macro del kernel que recorre todos los procesos activos.

Cada iteración entrega un puntero a una estructura task\_struct.

### Impresión de Procesos

```
seq_printf(m,  
    "%-10d %-25s %-15s\n",  
    proceso->pid,
```



```
proceso->comm,  
estado_proceso(proceso->__state));
```

Muestra:

- PID del proceso.
- Nombre del proceso.
- Estado traducido a texto.

### **Función abrir\_kmonitor()**

```
static int abrir_kmonitor(struct inode *inode,  
                          struct file *file)
```

Se ejecuta cuando un usuario abre el archivo dentro de /proc.

```
return single_open(file,  
                   mostrar_kmonitor,  
                   NULL);
```

Conecta la lectura del archivo con la función mostrar\_kmonitor().

### **Estructura proc\_ops**

```
static const struct proc_ops operaciones_proc
```

Define las operaciones que pueden realizarse sobre el archivo virtual.

```
.proc_open = abrir_kmonitor
```

Asocia la apertura del archivo.

```
.proc_read = seq_read
```

Permite la lectura secuencial.



```
.proc_lseek = seq_lseek
```

Permite desplazarse dentro del archivo.

```
.proc_release = single_release
```

Libera recursos al cerrar el archivo.

### **Función iniciar\_kmonitor()**

```
static int __init iniciar_kmonitor(void)
```

Función ejecutada automáticamente cuando se carga el módulo.

```
entrada_proc = proc_create(...)
```

Crea el archivo:

```
/proc/kmonitor_Grupo_10
```

```
if (!entrada_proc)
```

Verifica si la creación fue exitosa.

```
return -ENOMEM;
```

Retorna un error si no existe memoria suficiente.

```
printk(KERN_INFO
```

```
    "kmonitor_proyecto: modulo cargado\n");
```

Registra en el log del kernel que el módulo fue cargado correctamente.

### **Función salir\_kmonitor()**

```
static void __exit salir_kmonitor(void)
```





Función ejecutada cuando el módulo es descargado.

```
proc_remove(entrada_proc);
```

Elimina el archivo creado en /proc.

```
printk(KERN_INFO  
        "kmonitor_proyecto: modulo eliminado\n");
```

Registra la descarga del módulo en el log del kernel.

## **Macros Finales**

```
module_init(iniciar_kmonitor);
```

Indica cuál será la función de inicio del módulo.

```
module_exit(salir_kmonitor);
```

Indica cuál será la función de limpieza del módulo.

## **Archivo kmonitor.h**

Este archivo funciona como cabecera principal del proyecto.

Contiene:

- Bibliotecas del kernel.
- Declaraciones comunes.
- Constantes globales.

La constante más importante es:

```
#define NOMBRE_PROC "kmonitor_Grupo_10"
```



Esta constante define el nombre del archivo que aparecerá dentro del directorio /proc.

## **Archivo Makefile**

El archivo Makefile automatiza la compilación del módulo.

```
obj-m += kmonitor.o
```

Indica que el archivo kmonitor.c debe compilarse como módulo del kernel.

all:

Objetivo principal de compilación.

```
make -C /lib/modules/$(shell uname -r)/build M=$(shell pwd) modules
```

Invoca el sistema de compilación oficial del kernel Linux.

clean:

Objetivo utilizado para limpieza.

```
make -C /lib/modules/$(shell uname -r)/build M=$(shell pwd) clean
```

Elimina todos los archivos generados durante la compilación.

## **Explicación de las Estructuras de Datos del Kernel Utilizadas**

### **Introducción**

Uno de los principales objetivos del proyecto K-Monitor es interactuar directamente con las estructuras internas del Kernel Linux para obtener información sobre el



estado del sistema. Para lograrlo, el módulo utiliza diversas estructuras de datos y macros proporcionadas por el núcleo del sistema operativo.

Estas estructuras permiten acceder a información de procesos, memoria y sistemas de archivos virtuales de manera segura y eficiente.

## 1. Estructura `task_struct`

### Descripción

La estructura `task_struct` es una de las estructuras más importantes del Kernel Linux. Representa un proceso o tarea dentro del sistema operativo.

Cada vez que un proceso es creado, el kernel genera una instancia de esta estructura para almacenar toda la información relacionada con dicho proceso.

La tabla global de procesos del sistema está formada por múltiples estructuras `task_struct` enlazadas entre sí.

### Función en K-Monitor

En el proyecto K-Monitor se utiliza para obtener:

- PID del proceso.
- Nombre del proceso.
- Estado actual del proceso.

### Declaración utilizada

```
struct task_struct *proceso;
```

Esta declaración crea un puntero que permitirá recorrer cada proceso existente dentro del sistema.



## **Campos utilizados**

### **pid**

proceso->pid

Representa el identificador único del proceso.

Cada proceso posee un PID diferente asignado por el kernel.

Ejemplos:

#### **Proceso PID**

|         |      |
|---------|------|
| init    | 1    |
| systemd | 1    |
| bash    | 1250 |
| firefox | 4321 |

### **comm**

proceso->comm

Almacena el nombre corto del proceso.

Ejemplos:

- bash
- systemd
- sshd
- firefox
- chrome

Este campo facilita la identificación rápida de procesos activos.



**\_\_state**

proceso->\_\_state

Contiene el estado interno actual del proceso.

K-Monitor utiliza este valor para traducirlo a un texto comprensible para el usuario.

## **2. Macro for\_each\_process()**

### **Descripción**

La macro for\_each\_process() es una herramienta proporcionada por Linux para recorrer toda la lista de procesos activos del sistema.

Internamente recorre la lista enlazada global mantenida por el scheduler del kernel.

### **Sintaxis**

```
for_each_process(proceso)
{
    ...
}
```

### **Funcionamiento**

Cada iteración obtiene un puntero a una estructura task\_struct.

El recorrido continúa hasta visitar todos los procesos activos.

### **Uso dentro de K-Monitor**

```
for_each_process(proceso)
{
```



```
seq_printf(...);  
}
```

La macro permite mostrar:

- PID
- Nombre
- Estado

de todos los procesos existentes.

### **Ventajas**

- Evita recorrer manualmente listas enlazadas.
- Es segura.
- Es eficiente.
- Forma parte de las interfaces oficiales del kernel.

## **3. Estructura sysinfo**

### **Descripción**

La estructura sysinfo contiene estadísticas generales sobre la memoria y recursos del sistema.

Es utilizada por el kernel para proporcionar información global del estado actual de la máquina.

### **Declaración**

```
struct sysinfo info_sistema;
```

### **Obtención de datos**

```
si_meminfo(&info_sistema);
```



La función llena automáticamente la estructura con la información disponible.

### **Campos utilizados**

#### **totalram**

info\_sistema.totalram

Representa la cantidad total de memoria RAM instalada.

#### **freeram**

info\_sistema.freeram

Representa la cantidad de memoria RAM disponible.

### **Aplicación en K-Monitor**

Con estos valores se calcula:

$\text{memoria\_usada} = \text{memoria\_total} - \text{memoria\_libre};$

Posteriormente:

$\text{porcentaje\_uso} =$   
 $(\text{memoria\_usada} * 100) / \text{memoria\_total};$

Esto permite mostrar el porcentaje de utilización de memoria en tiempo real.

## **4. Sistema seq\_file**

### **Descripción**

El subsistema seq\_file es una interfaz del kernel diseñada para generar archivos virtuales dentro de /proc de manera eficiente.



Permite producir grandes cantidades de texto sin preocuparse por la administración manual de buffers.

## **Ventajas**

- Evita desbordamientos de memoria.
- Simplifica la programación.
- Es ampliamente utilizada por módulos del kernel.

## **Función seq\_printf()**

### **Sintaxis**

`seq_printf(...)`

Funciona de forma similar a `printf()` en lenguaje C.

Su diferencia es que escribe directamente dentro del buffer del archivo virtual.

### **Ejemplo**

```
seq_printf(m,  
    "Memoria total: %lu KB\n",  
    memoria_total);
```

Esta instrucción genera la salida visible para el usuario.

## **5. Estructura proc\_ops**

### **Descripción**

La estructura `proc_ops` define las operaciones permitidas sobre un archivo ubicado dentro del sistema `/proc`.

Actúa como una tabla de funciones que el kernel ejecutará cuando un usuario interactúe con el archivo.





## **Declaración utilizada**

```
static const struct proc_ops operaciones_proc
```

## **Operaciones implementadas**

### **proc\_open**

```
.proc_open = abrir_kmonitor
```

Se ejecuta cuando el archivo es abierto.

### **proc\_read**

```
.proc_read = seq_read
```

Permite leer información desde el archivo virtual.

### **proc\_lseek**

```
.proc_lseek = seq_lseek
```

Permite mover el cursor dentro del archivo.

### **proc\_release**

```
.proc_release = single_release
```

Libera recursos cuando el archivo es cerrado.

## **6. Estructura proc\_dir\_entry**

### **Descripción**

Representa una entrada dentro del sistema de archivos virtual /proc.



Cada archivo o directorio creado dentro de /proc posee una estructura de este tipo.

### **Declaración utilizada**

```
static struct proc_dir_entry *entrada_proc;
```

### **Creación**

```
entrada_proc =  
proc_create(  
NOMBRE_PROC,  
0444,  
NULL,  
&operaciones_proc);
```

Esta llamada crea el archivo:

```
/proc/kmonitor_Grupo_10
```

### **Eliminación**

```
proc_remove(entrada_proc);
```

Elimina completamente la entrada creada cuando el módulo es descargado.

### **Relación Entre Todas las Estructuras**

El funcionamiento conjunto de las estructuras utilizadas por K-Monitor puede resumirse de la siguiente forma:

1. El usuario ejecuta:

```
cat /proc/kmonitor_Grupo_10
```

2. La estructura proc\_ops recibe la solicitud.
3. Se ejecuta la función abrir\_kmonitor().



4. El sistema seq\_file prepara el buffer de salida.
5. La estructura sysinfo obtiene información de memoria.
6. La macro for\_each\_process() recorre todos los procesos.
7. Cada proceso es representado mediante una estructura task\_struct.
8. La información es enviada al usuario mediante seq\_printf().
9. Al descargar el módulo, proc\_dir\_entry elimina la entrada creada en /proc.

## Conclusión

Las estructuras task\_struct, sysinfo, proc\_dir\_entry, proc\_ops y el sistema seq\_file constituyen componentes fundamentales del Kernel Linux. Su utilización en K-Monitor permite acceder directamente a información crítica del sistema, demostrando cómo un módulo cargable puede interactuar con los mecanismos internos del kernel para monitorear memoria y procesos en tiempo real de forma segura y eficiente.



## CONCLUSIONES GENERALES

1. Se desarrolló satisfactoriamente un Módulo Cargable del Kernel capaz de obtener información sobre la memoria y los procesos activos del sistema operativo Linux.
2. Se comprobó el funcionamiento del sistema de archivos virtual /proc como mecanismo de comunicación entre el espacio del kernel y el espacio de usuario.
3. Se aplicaron conocimientos relacionados con la estructura `task_struct`, la gestión de procesos y la administración de memoria dentro del kernel Linux.
4. La utilización de la macro `for_each_process()` permitió recorrer eficientemente la tabla global de procesos administrada por el sistema operativo.
5. La función `si_meminfo()` facilitó la obtención de estadísticas de memoria en tiempo real, permitiendo calcular el porcentaje de utilización del sistema.
6. El proyecto fortaleció la comprensión de la arquitectura interna de Linux y del funcionamiento de los módulos cargables del kernel.
7. La implementación realizada demuestra cómo es posible desarrollar herramientas de monitoreo directamente dentro del kernel sin modificar permanentemente el sistema operativo.



## RECOMENDACIONES

1. Realizar las pruebas del módulo dentro de una máquina virtual para evitar afectar el sistema operativo principal.
2. Verificar siempre la compatibilidad entre la versión del kernel y los encabezados de desarrollo instalados.
3. Utilizar adecuadamente las funciones de limpieza para evitar fugas de memoria o referencias inválidas dentro del kernel.
4. Documentar ampliamente el código fuente para facilitar futuras modificaciones y tareas de mantenimiento.
5. Implementar mecanismos adicionales de monitoreo como uso de CPU, estadísticas de red y consumo de memoria por proceso para futuras versiones del proyecto.

## Anexo

Video: [https://drive.google.com/file/d/1GR5nDVU32jF8MbyLYutdxqV-T2BOFkC6/view?usp=drive\\_link](https://drive.google.com/file/d/1GR5nDVU32jF8MbyLYutdxqV-T2BOFkC6/view?usp=drive_link)

GitHub: [https://github.com/myantoniob/k\\_monitor\\_G10.git](https://github.com/myantoniob/k_monitor_G10.git)



## **BIBLIOGRAFÍA**

Love, R. (2010). Linux Kernel Development. Addison-Wesley Professional.

Bovet, D., & Cesati, M. (2005). Understanding the Linux Kernel. O'Reilly Media.

The Linux Kernel Documentation Project. Kernel Documentation.

Kerrisk, M. (2010). The Linux Programming Interface. No Starch Press.

Silberschatz, A., Galvin, P., & Gagne, G. (2018). Operating System Concepts. Wiley.

Linux Manual Pages Project. Linux Programmer's Manual.